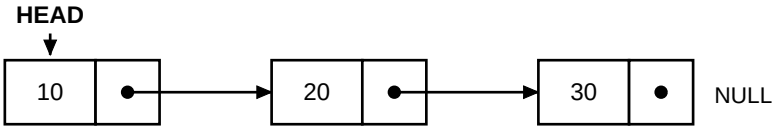
Linked List - Simple Notes

Data Structures (PBC2TE302) | CO1 | Singly, Doubly, Header and Circular Linked Lists with algorithms

1. What is a Linked List?

A **linked list** is a chain of boxes called **nodes**. Each node stores some **data** and the **address of the next node**. The nodes can be anywhere in memory; the addresses (links) connect them in the correct order.

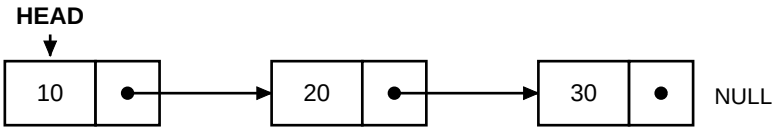


- **head** is a pointer that holds the address of the first node.
- The **last node** has NULL in its next part (it points to nothing).
- If head = NULL, the list is **empty**.
- The size grows and shrinks while the program runs (dynamic).
- Insertion and deletion are easy - we only change the links, no shifting like in an array.

Notation used below: p->next means "the next part of the node that p points to". **malloc()** creates a new node, **free()** removes a node from memory.

2. Singly Linked List

Each node has **two parts: DATA and NEXT**. We can move only in the **forward** direction, from head to the last node.




```
struct node {
    int data;
    struct node *next;
};
```

Algorithms (singly linked list)

Traversal (display all nodes)

Step 1: Set temp = head.

Step 2: Repeat while temp is not NULL: print temp->data and set temp = temp->next.

Step 3: Stop.

Insert at the beginning

Idea: new node points to the old first node, then head points to the new node.

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: Set newnode->next = head.

Step 3: Set head = newnode.

Step 4: Stop.

Insert at the end

Step 1: Create a new node (newnode), put the value in newnode->data and set newnode->next = NULL.

Step 2: If head is NULL, set head = newnode and stop.

Step 3: Set temp = head. Repeat while temp->next is not NULL: temp = temp->next. (Now temp is the last node.)

Step 4: Set temp->next = newnode.

Step 5: Stop.

Insert at a given position (pos > 1)

Idea: go to the node just before the position, then link the new node in between.

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: Set temp = head. Move temp forward (pos - 2) times. If temp becomes NULL, print "Invalid position" and stop.

Step 3: Set newnode->next = temp->next.

Step 4: Set temp->next = newnode. (Do step 3 before step 4.)

Step 5: Stop.

Delete from the beginning

Idea: remember the first node, move head to the second node, then free the old first node.

Step 1: If head is NULL, print "List is empty (Underflow)" and stop.

Step 2: Set temp = head.

Step 3: Set head = head->next.

Step 4: Free temp (free(temp)).

Step 5: Stop.

Delete from the end

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is NULL (only one node), free head, set head = NULL and stop.

Step 3: Set temp = head. Repeat while temp->next->next is not NULL: temp = temp->next. (Now temp is the second last node.)

Step 4: Free temp->next (the last node).

Step 5: Set temp->next = NULL.

Step 6: Stop.

Delete a node with a given value (key)

Idea: keep a pointer to the node before the one to be deleted, and skip over the deleted node.

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->data == key, delete the first node (as in "Delete from the beginning") and stop.

Step 3: Set prev = head and curr = head->next.

Step 4: Repeat while curr is not NULL and curr->data is not key: prev = curr and curr = curr->next.

Step 5: If curr is NULL, print "Key not found" and stop.

Step 6: Set prev->next = curr->next.

Step 7: Free curr.

Step 8: Stop.

Search for a value (key)

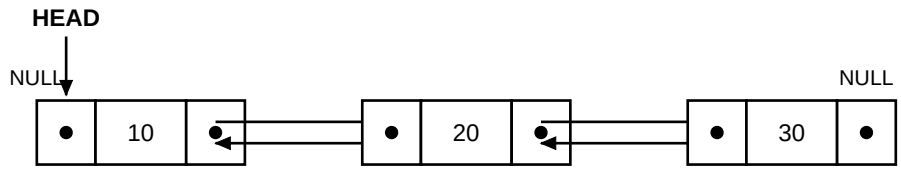
Step 1: Set temp = head and pos = 1.

Step 2: Repeat while temp is not NULL: if temp->data == key, print "Found at position pos" and stop; otherwise set temp = temp->next and pos = pos + 1.

Step 3: Print "Key not found". Stop.

3. Doubly Linked List

Each node has **three parts: PREV, DATA and NEXT**. PREV points to the previous node and NEXT points to the next node. We can move **both forward and backward**. The PREV of the first node and the NEXT of the last node are NULL.




```
struct dnode {
    struct dnode *prev;
    int data;
    struct dnode *next;
};
```

- **Good:** can be traversed in both directions; deleting a node is easier because we already know its previous node.
- **Not so good:** every node needs extra memory for the second pointer.

Algorithms (doubly linked list)

Traversal forward and backward

Step 1: Forward: set temp = head and repeat while temp is not NULL: print temp->data and set temp = temp->next.

Step 2: Backward: go to the last node (keep moving temp = temp->next until temp->next is NULL).

Step 3: Repeat while temp is not NULL: print temp->data and set temp = temp->prev.

Step 4: Stop.

Insert at the beginning

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: Set newnode->prev = NULL and newnode->next = head.

Step 3: If head is not NULL, set head->prev = newnode.

Step 4: Set head = newnode.

Step 5: Stop.

Insert at the end

Step 1: Create a new node (newnode), put the value in newnode->data and set newnode->next = NULL.

Step 2: If head is NULL, set newnode->prev = NULL, head = newnode and stop.

Step 3: Set temp = head. Repeat while temp->next is not NULL: temp = temp->next.

Step 4: Set temp->next = newnode and newnode->prev = temp.

Step 5: Stop.

Insert after a given node (temp)

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: Set newnode->next = temp->next and newnode->prev = temp.

Step 3: If temp->next is not NULL, set temp->next->prev = newnode.

Step 4: Set temp->next = newnode.

Step 5: Stop.

Delete from the beginning

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: Set temp = head and head = head->next.

Step 3: If head is not NULL, set head->prev = NULL.

Step 4: Free temp.

Step 5: Stop.

Delete from the end

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is NULL (only one node), free head, set head = NULL and stop.

Step 3: Set temp = head. Repeat while temp->next is not NULL: temp = temp->next. (Now temp is the last node.)

Step 4: Set temp->prev->next = NULL.

Step 5: Free temp.

Step 6: Stop.

Delete a node with a given value (key)

Idea: connect the previous node and the next node to each other, then free the node.

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: Set temp = head. Repeat while temp is not NULL and temp->data is not key: temp = temp->next.

Step 3: If temp is NULL, print "Key not found" and stop.

Step 4: If temp is the first node, set head = temp->next. Otherwise set temp->prev->next = temp->next.

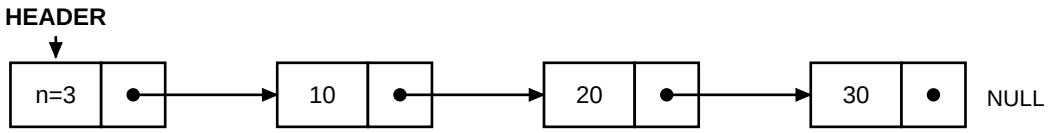
Step 5: If temp->next is not NULL, set temp->next->prev = temp->prev.

Step 6: Free temp.

Step 7: Stop.

4. Header Linked List

A **header linked list** has a special extra node at the very beginning called the **header node**. It does **not store real data**; it usually stores the **number of nodes** in the list. The real list starts after the header node. Because the header node is always there, the list is never completely empty, so insertion at the beginning needs no special case.



Insert at the beginning (after the header)

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: Set newnode->next = header->next.

Step 3: Set header->next = newnode.

Step 4: Increase the count: header->data = header->data + 1.

Step 5: Stop.

Delete the first real node

Step 1: If header->next is NULL, print "List is empty" and stop.

Step 2: Set temp = header->next.

Step 3: Set header->next = temp->next.

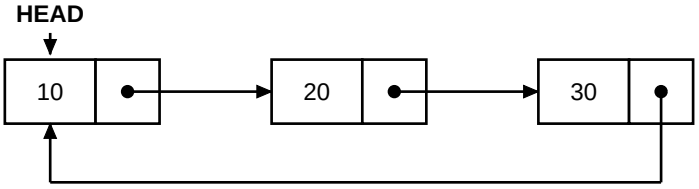
Step 4: Free temp.

Step 5: Decrease the count: header->data = header->data - 1.

Step 6: Stop.

5. Circular Singly Linked List

It is a singly linked list in which the **next of the last node points back to the first node** (not NULL). So the nodes form a circle and there is **no NULL** anywhere. We can reach every node starting from any node. Used in round-robin scheduling, music playlists, etc.



- The list is empty when head = NULL.
- The **last node** is the node whose next is head.
- While traversing, we stop when we come back to head (we cannot stop at NULL).

Algorithms (circular singly linked list)

Traversal

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: Set temp = head.

Step 3: Print temp->data and set temp = temp->next. Repeat this until temp becomes head again.

Step 4: Stop.

Insert at the beginning

Idea: the last node must also point to the new first node.

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: If head is NULL, set head = newnode, newnode->next = newnode (points to itself) and stop.

Step 3: Set temp = head. Repeat while temp->next is not head: temp = temp->next. (Now temp is the last node.)

Step 4: Set newnode->next = head.

Step 5: Set temp->next = newnode.

Step 6: Set head = newnode.

Step 7: Stop.

Insert at the end

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: If head is NULL, set head = newnode, newnode->next = newnode and stop.

Step 3: Set temp = head. Repeat while temp->next is not head: temp = temp->next.

Step 4: Set temp->next = newnode.

Step 5: Set newnode->next = head.

Step 6: Stop. (head does not change.)

Delete from the beginning

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is head (only one node), free head, set head = NULL and stop.

Step 3: Set last = head. Repeat while last->next is not head: last = last->next.

Step 4: Set temp = head and head = head->next.

Step 5: Set last->next = head.

Step 6: Free temp.

Step 7: Stop.

Delete from the end

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is head (only one node), free head, set head = NULL and stop.

Step 3: Set temp = head. Repeat while temp->next->next is not head: temp = temp->next. (Now temp is the second last node.)

Step 4: Set last = temp->next.

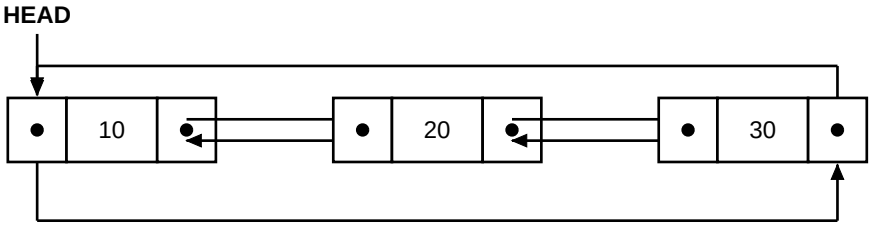
Step 5: Set temp->next = head.

Step 6: Free last.

Step 7: Stop.

6. Circular Doubly Linked List

It is a doubly linked list in which the **next of the last node points to the first node** and the **prev of the first node points to the last node**. We can move in both directions in a continuous circle. The last node is always **head->prev**, so we can reach it directly without a loop.



Algorithms (circular doubly linked list)

Insert at the beginning

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: If head is NULL, set head = newnode, newnode->next = newnode, newnode->prev = newnode and stop.

Step 3: Set last = head->prev.

Step 4: Set newnode->next = head and newnode->prev = last.

Step 5: Set last->next = newnode and head->prev = newnode.

Step 6: Set head = newnode.

Step 7: Stop.

Insert at the end

Step 1: Create a new node (newnode) and put the value in newnode->data.

Step 2: If head is NULL, set head = newnode, newnode->next = newnode, newnode->prev = newnode and stop.

Step 3: Set last = head->prev.

Step 4: Set newnode->next = head and newnode->prev = last.

Step 5: Set last->next = newnode and head->prev = newnode.

Step 6: Stop. (head does not change.)

Delete from the beginning

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is head (only one node), free head, set head = NULL and stop.

Step 3: Set last = head->prev and temp = head.

Step 4: Set head = head->next.

Step 5: Set head->prev = last and last->next = head.

Step 6: Free temp.

Step 7: Stop.

Delete from the end

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: If head->next is head (only one node), free head, set head = NULL and stop.

Step 3: Set last = head->prev.

Step 4: Set last->prev->next = head.

Step 5: Set head->prev = last->prev.

Step 6: Free last.

Step 7: Stop.

Traversal (forward)

Step 1: If head is NULL, print "List is empty" and stop.

Step 2: Set temp = head.

Step 3: Print temp->data and set temp = temp->next. Repeat until temp becomes head again.

Step 4: Stop. (For backward traversal start from head->prev and use prev links.)

7. Quick Summary

- **Singly:** node = data + next. Moves forward only. Last node points to NULL.
- **Doubly:** node = prev + data + next. Moves both ways. Needs more memory.
- **Header:** extra first node (header) that stores the count; real data starts after it.
- **Circular singly:** last node points to the first node. No NULL. Stop traversal when temp == head.
- **Circular doubly:** last->next = first and first->prev = last. Moves both ways in a circle.
- Always check **head == NULL** before deleting (underflow) and handle the **one node** case separately.
- While inserting, **set the new node's links first**, then change the old links. While deleting, **keep the node in temp** so that it can be freed.
- Insert/delete at the beginning is O(1) in singly and doubly lists; insert/delete at the end needs a traversal O(n) unless we keep a tail pointer.

Practice questions

1. What is a linked list? Compare it with an array.
2. Explain singly, doubly and circular linked lists with diagrams.
3. Write algorithms to insert a node at the beginning and at the end of a singly linked list.
4. Write algorithms to delete a node from the beginning and from the end of a singly linked list.
5. Write an algorithm to insert and delete a node in a doubly linked list.
6. What is a header linked list? Explain with a diagram.
7. Write an algorithm for insertion at the beginning in a circular linked list.